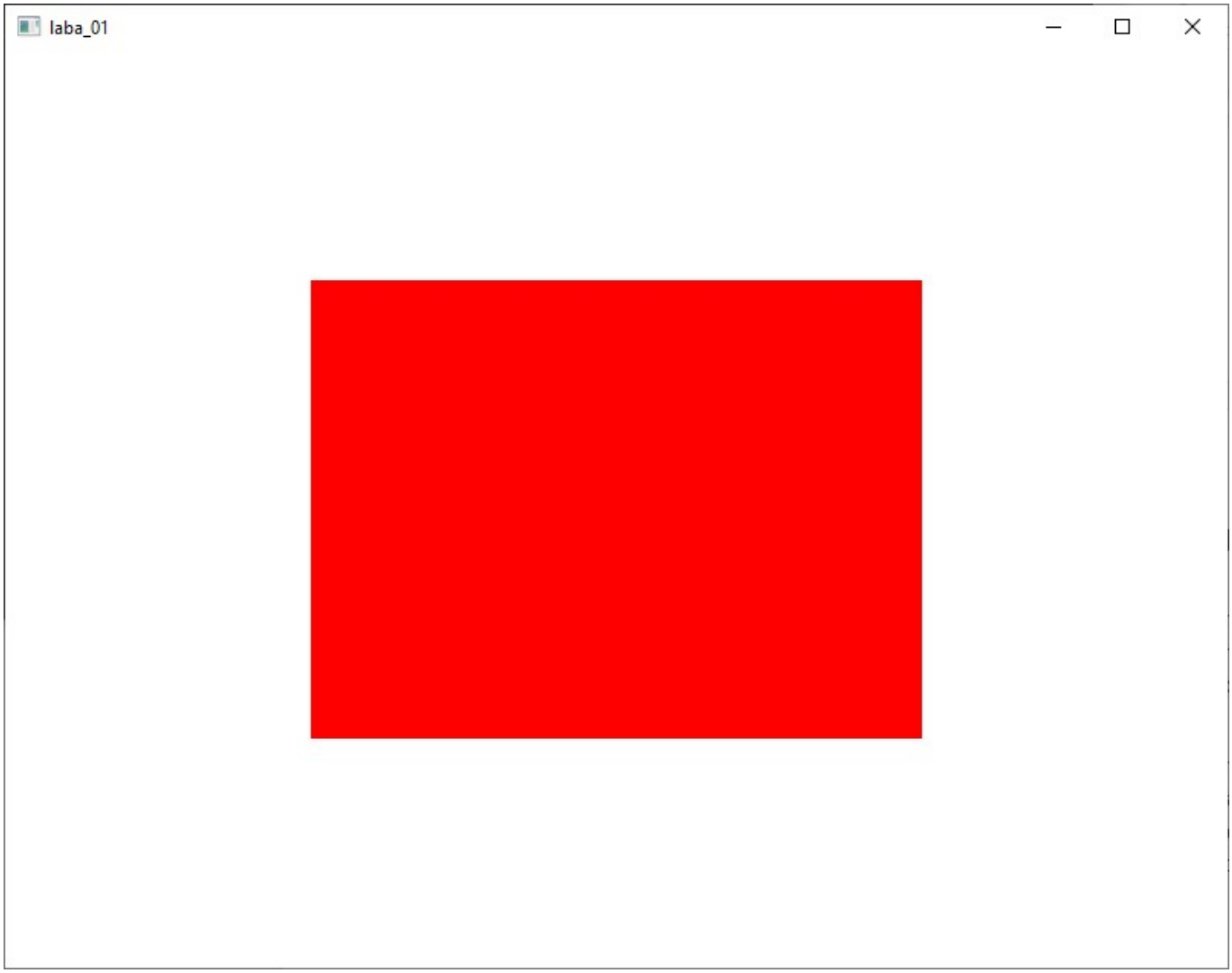
Лабораторная работа №01

Загрузка и использование шейдера.

Целью лабораторной работы №01 является изучение функций OpenGL для создания шейдерных программ, загрузки текстов шейдеров, их компилирования и линковки, а также изучение функций для использования полученного шейдера с целью вывода объектов на экран.



Кроме того, в данной лабораторной работе необходимо продемонстрировать умения, полученные при реализации работ предыдущего семестра. В частности, необходимо продемонстрировать умение создавать проект, подключать необходимые библиотеки, создавать оконные приложения, используя библиотеку freeglut, создавать модули и классы, работать с файлами и некоторыми контейнерами из стандартной библиотеки шаблонов C++.

При создании оконного приложения необходимо использовать OpenGL версии 3.3 или выше, поэтому метод инициализации контекста рендеринга и создания окна будет немного отличаться от того, что был использован в лабораторных работах предыдущего курса.

Порядок выполнения лабораторной работы.

Основная цель данной лабораторной работы – это написание класса, позволяющего удобным способом работать с шейдерами, в частности: загружать вершинный и фрагментный шейдер из внешних файлов, объединять вершинный и фрагментный шейдеры в одну шейдерную программу, а также выбирать шейдер для использования.

Для облегчения начала работы с шейдерами в приложении приводится как базовая версия основной программы, так и тексты шейдеров, которые необходимо использовать для начального тестирования работоспособности реализуемого класса. Структура класса для работы с шейдерами (Shader) также приводится в методических указаниях, однако реализацию класса необходимо написать самостоятельно.

Процесс написания программы рекомендуется разбить на несколько относительно независимых этапов, что позволит в каждый момент времени сосредоточиться на решении только одной конкретной задачи и не даст впасть в уныние от осознания того, насколько большими могут быть некоторые проекты.

Перед началом выполнения этапа может потребоваться освежить знания в некоторых областях, например, заново просмотреть синтаксис создания классов или методы работы с файлами в C++. При изучении указанных тем, для их лучшего усвоения, рекомендуется писать демонстрационные примеры, чтобы убедиться в правильности понимания всех описываемых понятий и ключевых моментов.

При реализации каждого этапа необходимо жестко контролировать, что данный этап выполняет все, что от него требуется и работает корректно. Для проверки работоспособности этапа рекомендуется тестировать его на нескольких вариантах входных данных, а также в процессе тестирования выводить контрольную информацию на консоль. Это позволит уменьшить количество возможных ошибок.

В завершении следует отметить, что если какой-либо этап оказался достаточно сложным или потребовал существенного количества времени на его реализацию, то после завершения этапа можно позволить себе небольшой отдых или даже перерыв.

Для облегчения разработки программы, первую лабораторную работу рекомендуется выполнять в соответствии со следующей последовательностью этапов:

1. **Создание проекта.**

   На основе примера, приведенного в приложении, необходимо создать новый проект, в котором бы создавалось окно с контекстом рендеринга OpenGL 3.3 или выше. Для создания окна используется библиотека freeGLUT, а для инициализации расширений библиотека glew. Обе библиотеки были рассмотрены в предыдущем курсе лекций. Здесь также рекомендуется подключить и другие библиотеки, которые будут использоваться в проекте чуть позже, в частности: glm, RapidJSON и DevIL.

2. **Создание класса для работы с шейдерами.**

   На втором этапе в отдельном модуле необходимо создать прототип класса для работы с шейдерами и подключить созданный модуль к проекту. Минимальная структура класса приведена в соответствующем разделе.

3. **Реализация всех методов класса Shader.**

   В заключении необходимо реализовать все методы класса Shader. Реализацию сложных методов, например, загрузку шейдера, в свою очередь также можно разбить на несколько этапов в соответствии с личными способностями обучающегося.

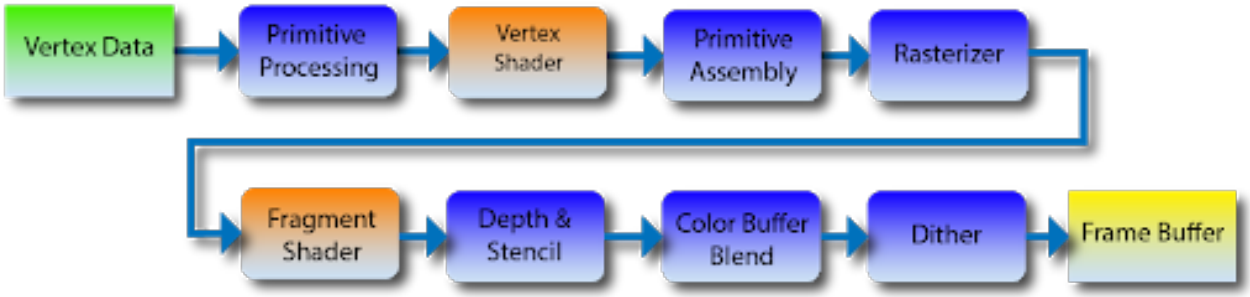
   Для тестирования класса требуется использовать шейдеры, приведенные в примере и в приложении к методическим указаниям, чтобы убедиться в том, что класс успешно загружает, компилирует и линкует шейдеры. При загрузке шейдеров необходимо проверять, что указанные файлы существуют, что компиляция прошла без ошибок, и что удалось выполнить линковку. Для отладки рекомендуется выводить диагностические сообщения на консоль.

Работа с шейдерами в OpenGL.

Шейдер (Shader) – это программа, определяющая процесс построения изображения на экране, выполняющаяся на видеокарте и замещающая собой некоторые ступени графического конвейера. В OpenGL различают понятия «шейдерная программа» и «шейдерный объект».

Шейдерный объект – это внутренняя структура данных OpenGL, которая представляет отдельно взятую программу, выполняющуюся на видеокарте и предназначенную для замещения только одной, конкретной ступени конвейера. Различают вершинный шейдерный объект (вершинный шейдер) и фрагментный шейдерный объект (фрагментный шейдер). Вершинный шейдер занимается преобразованием вершин, а фрагментный шейдер – вычислением цвета фрагмента. Также существуют и другие типы шейдеров – например: геометрический шейдер, шейдер тесселяции или вычислительный шейдер, однако они не являются обязательными и в данном курсе не рассматриваются. Шейдерные объекты не могут использоваться сами по себе, то есть нельзя вывести объект на экран, используя только вершинный или только фрагментный шейдерный объект.

Шейдерная программа – это внутренняя структура данных OpenGL, которая объединяет все шейдерные объекты, необходимые для вывода графического объекта (меша) на экран. В частности, шейдерная программа должна содержать как минимум один вершинный шейдерный объект и один фрагментный шейдерный объект, которые совместно образуют шейдерную программу. Остальные шейдерные объекты являются опциональными и в данной лабораторной работе не используются. Именно шейдерная программа (в дальнейшем, просто шейдер) используется для вывода моделей на экран. Следует отметить, что программа может оперировать сразу несколькими шейдерами и для каждого объекта использовать свой собственный шейдер.



Работа с шейдерами в OpenGL состоит из трех основных этапов:

1. Написание текста шейдера, что подразумевает определение алгоритма, по которому он будет функционировать, и данных, которые необходимо будет передать в шейдер для его функционирования. Как правило, шейдеры хранятся в виде внешних, по отношению к программе, текстовых файлов, что позволяет их легко просмотреть или отредактировать. В рамках данной лабораторной работы прилагается пример с файлами, содержащими текст шейдеров (Example.vsh и Example.fsh для вершинного и фрагментного шейдеров соответственно).

2. Загрузка шейдера в видеокарту (в OpenGL) и его компиляция средствами самой видеокарты. Загрузка шейдера выполняется, как правило, только один раз при инициализации приложения. В дальнейшем шейдер может использоваться столько раз, сколько потребуется. Если шейдер больше не нужен, его можно удалить.

3. Использование шейдера, то есть выбор шейдера в качестве текущего, передача в шейдер определенных параметров и последующий вывод графических объектов. Вывод графических объектов на экран подразумевает передачу в OpenGL всех необходимых вершин. При этом для каждой вершины видеокарта автоматически вызывает вершинный шейдер и передает ему все указанные атрибуты вершины. После растеризации для каждого полученного фрагмента автоматически вызывается фрагментный шейдер.

В рамках данной лабораторной работы наибольший интерес представляет именно процесс загрузки и компиляции шейдера. При этом следует отметить, что OpenGL имеет функции, которые отвечают только за передачу текста шейдера из оперативной памяти в видеокарту. Операция загрузки данных из внешнего файла в оперативную память осуществляется стандартными средствами C++ и не имеет отношение к OpenGL. Сам процесс создания шейдера (шейдерной программы) был подробно рассмотрен на лекции и может быть представлен следующей последовательностью шагов:

1. ***Создание и загрузка вершинного шейдерного объекта:***

   a. Создание вершинного шейдерного объекта с помощью функции glCreateShader. Данная функция возвращает идентификатор шейдерного объекта, который необходимо запомнить для последующего использования.

   b. Загрузка вершинного шейдера из внешнего файла в оперативную память. Данный этап выполняется стандартными способами, без использования функций OpenGL, поскольку он не имеет отношения к графической библиотеке.

   c. Передача текста шейдера в созданный шейдерный объект с помощью функции glShaderSource.

   d. Компиляция шейдерного объекта с помощью функции glCompileShader.

   e. Получения статуса компиляции и других параметров шейдера при необходимости (например, длины информационного сообщения об ошибках). Данная операция выполняется с помощью функции glGetShaderiv.

   f. Получение сообщения об ошибке – выполняется с помощью функции glGetShaderInfoLog.

2. ***Создание и загрузка фрагментного шейдерного объекта.***

   Выполняется аналогично созданию и загрузке вершинного шейдерного объекта.

3. ***Создание, инициализация и линковка шейдерной программы.***

   Шейдерная программа должна включать в себя один вершинный и один фрагментный шейдер. При этом вершинный и фрагментный шейдер должны быть согласованны, то есть выходные переменные вершинного шейдера (out) должны совпадать по количеству и структуре со входными переменными фрагментного шейдера (in). Данная операция выполняется в соответствии со следующим планом:

   a. Создание шейдерной программы с помощью функции glCreateProgram. Данная функция возвращает идентификатор шейдерной программы, который необходимо запомнить для последующего использования. Именно данный идентификатор будет использоваться при выборе этого шейдера для вывода последующих объектов.

   b. Присоединение к шейдерной программе вершинного шейдерного объекта с помощью функции glAttachShader.

   c. Присоединение к шейдерной программе фрагментного шейдерного объекта с помощью функции glAttachShader.

   d. Линковка шейдерной программы, которая выполняется с помощью функции glLinkProgram. Следует отметить, что линковка возможна только после того, как соответствующие шейдерные объекты были загружены и успешно скомпилированы.

   e. Получение статуса линковки с помощью функции glGetProgramiv.

   f. При необходимости можно получить сообщения об ошибке с помощью функции glGetProgramInfoLog.

Процесс использования шейдерной программы (шейдера) для вывода объектов на экран можно разделить на следующие этапы (при этом не все этапы относятся непосредственно к шейдеру):

1. ***Выбор шейдера в качестве текущего.***

   Для выбора шейдера в качестве текущего используется функция glUseProgram. Параметром функции является идентификатор шейдерной программы, полученный ранее с помощью функции glCreateProgram. Таким образом, каждая шейдерная программа имеет свой собственный идентификатор, поэтому можно использовать разные шейдеры для разных объектов. На данном этапе используется только один шейдер.

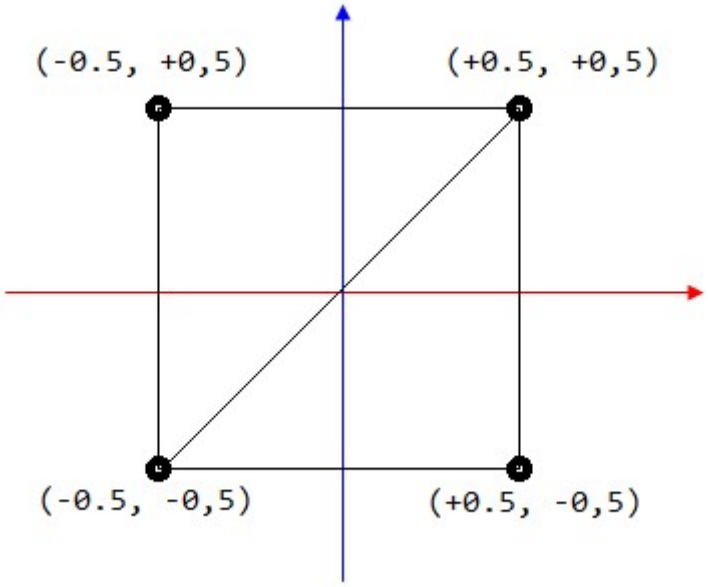
2. ***Установка некоторых внутренних переменных шейдера.***

   Данные переменные называются uniform-переменными. Uniform-переменная - это переменная, объявленная в шейдере и являющаяся общей для всего объекта. Значение uniform-переменной устанавливается из основной программы и сохраняется внутри указанной шейдерной программы (во внутренней структуре OpenGL). Таким образом, разные шейдерные программы могут иметь принципиально разный состав uniform-переменных или одинаковый состав uniform-переменных, но с разным значением каждой из них.

   Процесс работы с uniform-переменными будет рассмотрен в следующей лабораторной работе. В данной лабораторной работе установка uniform-переменных не производится.

3. ***Вывод полигонов.***

   Вывод полигонов подразумевает передачу в OpenGL всех вершин с указанием их атрибутов. При этом необходимо указать, в какие переменные вершинного шейдера какие атрибуты вершин будут передаваться. То есть необходимо определить соответствие между атрибутами вершин, которые хранятся в буфере OpenGL, и входными переменными вершинного шейдера (глобальными переменными, объявленными в вершинном шейдере с модификатором *in)*. В приложении 1, которое находится в конце данных методических указаний, приведен процесс вывода полигонов, обладающих всего одним атрибутом – двухмерными геометрическими координатами (x, y). При этом значение атрибутов определяется в соответствии с рисунком:



Класс для работы с шейдером.

Для удобства работы с шейдером, его необходимо оформить как отдельный класс, структура которого приведена ниже. Назначение основных полей и методов понятно по их названию, именам параметров и приведенным комментариям. Следует обратить внимание, что для реализации некоторых действий были выделены отдельные, вспомогательные, приватные методы.

```
// КЛАСС ДЛЯ РАБОТЫ С ШЕЙДЕРОМ
class Shader
{
public:
	// загрузка шейдера из внешних файлов
	bool load(std::string veftexShaderFilename, std::string fragmentShaderFilename);

	// выбор шейдера в качестве текущего
	void activate();

	// отключение шейдера
	static void deactivate();

private:
	// создание шейдерного объекта указанного типа
	// и загрузка исходного текста шейдера из указанного файла
	GLuint createShaderObject(GLenum shaderType, std::string filename);

private:
	// шейдерная программа (шейдер)
	GLuint program;

};
```

При необходимости разрешается (и даже приветствуется) добавление в класс новых полей и методов, если это упрощает работу с шейдерами и не приводит к отказу от принципа единственной ответственности (single-responsibility principle).

Задание к лабораторной работе.

В лабораторной работе №1 необходимо:

1. Создать проект, подключив к нему следующие библиотеки: freeglut, glew, glm, DevIL и RapidJSON. Библиотеки должны быть подключены как для debug, так и для release версий.
2. В качестве основной программы необходимо использовать код, приведенный в приложении 1.
3. Реализовать класс Shader для работы с шейдером с описанной выше структурой и оформить его в виде отдельного модуля. При необходимости разрешается добавлять новые поля и методы, если это является удобным, логичным и не нарушает принципа единственной ответственности.
4. Используя шейдеры из приложения 2, вывести на экран красный прямоугольник.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №1 необходимо:

1. Создать проект, подключив к нему следующие библиотеки: freeglut, glew, glm, DevIL и RapidJSON. Библиотеки должны быть подключены как для debug, так и для release версий.
2. В качестве основной программы необходимо использовать код, приведенный в приложении 1.
3. Реализовать класс Shader для работы с шейдером с описанной выше структурой и оформить его в виде отдельного модуля. При необходимости разрешается добавлять новые поля и методы, если это является удобным, логичным и не нарушает принципа единственной ответственности.
4. Используя шейдеры из приложения 2, вывести на экран красный прямоугольник.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		

Приложение 1: текст основной программы.

```
#include <windows.h>
#include "stdio.h"

#include "glew.h"
#include "GL/freeglut.h"

#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

#include "Shader.h"


// используемые пространства имен (для удобства)
using namespace glm;
using namespace std;


// используемый шейдер (пока только один)
Shader shader;

// функция для вывода квадрата с ребрами равными единице (от -0.5 до +0.5)
void drawObject();

// функция вызывается при перерисовке окна
// в том числе и принудительно, по команде glutPostRedisplay
void display()
{
	// отчищаем буфер кадра
	glClearColor (1.0, 1.0, 1.0, 1.0);
	glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

	// включаем тест глубины (на всякий случай)
	glEnable(GL_DEPTH_TEST);

	// активируем шейдер
	shader.activate();

	// выводим прямоугольник
	drawObject();

	// смена переднего и заднего буферов
	glutSwapBuffers();
}

// функция, вызываемая при изменении размеров окна
void reshape(int w,int h)
{
	// установить новую область просмотра, равную всей области окна
	glViewport(0, 0, w, h);
}

// функция вызывается, когда процессор простаивает, т.е. максимально часто
void simulation()
{
	// перерисовать окно
	glutPostRedisplay();
};
```

```
// основная функция
void main(int argc, char **argv)
{
	// инициализация библиотеки GLUT
	glutInit(&argc,argv);
	// инициализация дисплея (формат вывода)
	glutInitDisplayMode (GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH
                                | GLUT_STENCIL | GLUT_MULTISAMPLE);
	// требования к версии OpenGL (версия 3.3 без поддержки обратной совместимости)
	glutInitContextVersion(3, 3);
	glutInitContextProfile(GLUT_CORE_PROFILE);
	// устанавливаем верхний левый угол окна
	glutInitWindowPosition(300,100);
	// устанавливаем размер окна
	glutInitWindowSize(800,600);
	// создание окна
	glutCreateWindow("laba_01");

	// инициализация GLEW
	GLenum err = glewInit();
	if (GLEW_OK != err)
	{
	  fprintf(stderr, "Glew error: %s\n", glewGetErrorString(err));
	  return;
	}

	// определение текущей версии OpenGL
	printf("OpenGL Version = %s\n\n",glGetString(GL_VERSION));

	// загрузка шейдера
	shader.load("SHADER\\Example.vsh", "SHADER\\Example.fsh");

	// устанавливаем функцию, которая будет вызываться для перерисовки окна
	glutDisplayFunc(display);
	// устанавливаем функцию, которая будет вызываться при изменении размеров окна
	glutReshapeFunc(reshape);
	// устанавливаем функцию, которая вызывается всякий раз, когда процессор простаивает
	glutIdleFunc(simulation);
	// основной цикл обработки сообщений ОС
	glutMainLoop();
}
```

```
// функция для вывода квадрата с ребрами равными единице (от -0.5 до +0.5)
void drawObject()
{
	// переменные для вывода объекта (прямоугольника из двух треугольников)
	static bool init = true;
	static GLuint VAO_Index = 0;		// индекс VAO-буфера
	static GLuint VBO_Index = 0;		// индекс VBO-буфера
	static int VertexCount = 0;		// количество вершин

	// при первом вызове инициализируем VBO и VAO
	if (init) {
		init = false;
		// создание и заполнение VBO
		glGenBuffers(1, &VBO_Index);
		glBindBuffer(GL_ARRAY_BUFFER, VBO_Index);
		GLfloat	Verteces[] = {
			-0.5,	+0.5,
			-0.5,	-0.5,
			+0.5,	+0.5,
			+0.5,	+0.5,
			-0.5,	-0.5,
			+0.5,	-0.5
		};
		glBufferData(GL_ARRAY_BUFFER, sizeof(Verteces), Verteces, GL_STATIC_DRAW);

		// создание VAO
		glGenVertexArrays(1, &VAO_Index);
		glBindVertexArray(VAO_Index);
		// заполнение VAO
		glBindBuffer(GL_ARRAY_BUFFER, VBO_Index);
		glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 0, 0);
		glEnableVertexAttribArray(0);
		// "отвязка" буфера VAO, чтоб случайно не испортить
		glBindVertexArray(0);

		// указание количество вершин
		VertexCount = 6;
	}

	// выводим прямоугольник
	glBindVertexArray(VAO_Index);
	glDrawArrays(GL_TRIANGLES, 0, 6);
}
```

Приложение 2: текст вершинного и фрагментного шейдеров.

Текст вершинного шейдера:

```
#version 330 core

in vec2 vPosition;

void main()
{
	gl_Position = vec4(vPosition, 0, 1);
}
```

Текст фрагментного шейдера:

```
#version 330 core

out vec4 fragColor;

void main()
{
	fragColor = vec4(1.0, 0.0, 0.0, 0.0);
}
```